

# Apprentissage contrastif auto-supervisé pour le pré-entraînement visuel :

de combien d'annotations SimCLR nous dispense-t-il sur CIFAR-10 ?

KAMBIA Rafiatou • ALLAGLO Kossiko • AMEGNRAN Athanase

Master — Cours 14, Projets de recherche appliquée

Encadrant : TCHAYE-KONDI Jude, Ph.D. • Août 2026

Code et instructions de reproduction : <https://github.com/sunborndev/simclr-cifar10>

**Résumé.** Annoter des images coûte cher ; le pré-entraînement auto-supervisé promet d'exploiter les images non étiquetées pour réduire ce coût. Nous implémentons un cadre contrastif de type SimCLR en PyTorch, pré-entraînons un ResNet-18 sur les 50 000 images de CIFAR-10 *sans aucun label*, puis mesurons ce que ce pré-entraînement apporte lorsque 1 %, 10 % et 100 % des annotations sont disponibles. La comparaison est appariée : le fine-tuning de l'encodeur pré-entraîné et la baseline supervisée *from scratch* partagent la totalité de leurs hyperparamètres, seuls les poids initiaux diffèrent. Avec 450 images étiquetées, le pré-entraînement fait passer l'accuracy de test de 39,52 % à 72,10 %, soit +32,58 points ; avec 45 000 labels, les deux méthodes sont à parité. Un bras de contrôle à encodeur aléatoire gelé (25,66 %) établit que ce gain vient du pré-entraînement et non de l'architecture. Une étude d'ablation additive sur cinq combinaisons d'augmentations montre que les perturbations de couleur portent l'essentiel de la qualité de la représentation (+22,54 points à elles deux), tandis que le retournement horizontal et le flou gaussien n'ont aucun effet mesurable à cette échelle. Nous documentons enfin une dissociation nette entre la qualité de la tâche prétexte et celle de la représentation : la configuration qui minimise le mieux la perte NT-Xent produit la pire représentation. Un diagnostic mené sur les données seules, sans réseau de neurones, en donne le mécanisme chiffré : privé de distorsion de couleur, un simple histogramme de couleur retrouve la bonne image dans 51,9 % des cas parmi 2 000 candidates, contre 0,05 % au hasard — la tâche prétexte est à moitié résolue avant tout apprentissage.

**Mots-clés :** apprentissage auto-supervisé, apprentissage contrastif, SimCLR, NT-Xent, étude d'ablation, régime de faible supervision.

## 1 Introduction

**Contexte.** Les réseaux de neurones profonds appliqués à la vision supposent généralement l'existence d'un jeu de données annoté en quantité suffisante. En pratique, cette hypothèse est la plus coûteuse du pipeline : l'annotation demande du temps humain, parfois une expertise rare (imagerie médicale, télédétection), et devient impraticable à l'échelle des flux d'images réellement disponibles. Les images *non étiquetées*, en revanche, sont abondantes et quasi gratuites.

**Problématique.** L'apprentissage auto-supervisé (*self-supervised learning*, SSL) propose de construire un signal d'apprentissage à partir des données elles-mêmes, sans intervention humaine. Dans le cadre contrastif de SimCLR [1], ce signal est simple : deux vues augmentées de la *même* image doivent produire des représentations proches, et se distinguer des vues des autres images du lot. Le modèle apprend ainsi ce qui, dans une image, résiste aux déformations que l'on lui inflige — et c'est précisément ce qui définit son contenu sémantique.

La question que nous traitons est quantitative et non conceptuelle :

**Question de recherche.** À budget de calcul strictement égal, de combien d'images annotées un pré-entraînement contrastif dispense-t-il un classifieur CIFAR-10 ? Et quelles augmentations de données ce pré-entraînement exige-t-il pour produire une représentation utile ?

**Contributions.** Ce travail apporte les six éléments suivants.

- (i) Une implémentation de bout en bout du cadre SimCLR en PyTorch — construction des paires, encodeur partagé, tête de projection, perte NT-Xent — écrite à partir des équations du papier et non reprise d’une bibliothèque de haut niveau (section 4).
- (ii) Une comparaison *appariée* sur trois régimes d’annotation (450, 4 500, 45 000 labels) opposant quatre bras : fine-tuning du pré-entraîné, évaluation linéaire du pré-entraîné gelé, baseline supervisée *from scratch*, et un témoin à encodeur aléatoire gelé (section 6).
- (iii) Une étude d’ablation additive des augmentations en cinq configurations, qui isole la contribution de chaque brique (section 5).
- (iv) La mise en évidence mesurée d’une dissociation entre performance sur la tâche prétexte et qualité de la représentation apprise (section 5.4).
- (v) Un diagnostic sans réseau de neurones qui *quantifie* le raccourci de couleur responsable de cette dissociation : un simple histogramme de couleur résout la moitié de la tâche prétexte lorsque les augmentations de couleur sont absentes (section 5.5).
- (vi) Une application web de démonstration projetant les représentations apprises en 2D, colorées par une classe réelle jamais vue à l’entraînement (section 7).

## 2 Travaux liés

**SimCLR.** Chen *et al.* [1] formalisent le cadre que nous reprenons : deux augmentations tirées de la même famille  $\mathcal{T}$  produisent une paire positive, un encodeur partagé  $f(\cdot)$  en extrait des représentations, une tête de projection  $g(\cdot)$  les envoie dans l’espace où la perte contrastive s’applique, et  $g(\cdot)$  est jetée après le pré-entraînement. Leurs deux résultats les plus cités nous concernent directement : la *composition* d’augmentations compte davantage que chaque augmentation prise isolément — la paire recadrage + distorsion de couleur étant critique — et la performance croît fortement avec la taille du lot, donc avec le nombre de négatifs disponibles. Ce second point définit notre principale contrainte matérielle (section 8).

**MoCo.** He *et al.* [2] attaquent exactement cette contrainte. Plutôt que de tirer les négatifs du lot courant, MoCo maintient une *file d’attente* de représentations calculées aux itérations précédentes par un encodeur à moyenne mobile (momentum). Le nombre de négatifs est ainsi découplé de la taille du lot que la mémoire GPU autorise. C’est la référence naturelle pour tout projet contraint par un GPU unique.

**Travaux complémentaires : se passer de négatifs.** Deux familles postérieures suppriment le besoin de négatifs explicites, et donc la dépendance au lot. BYOL [3] n’utilise que des paires positives : un réseau «en ligne» apprend à prédire la sortie d’un réseau «cible» dont les poids sont une moyenne mobile des siens, l’asymétrie entre les deux branches empêchant l’effondrement des représentations. DINO [4] pousse cette idée sous forme de distillation auto-supervisée sans étiquettes, en s’appuyant sur des transformeurs de vision, et fait émerger des cartes d’attention qui segmentent les objets sans supervision.

**Positionnement de notre travail.** Nous n’implémentons ni MoCo, ni BYOL, ni DINO : le sujet demande un SimCLR simplifié, et notre budget de calcul est d’environ sept heures de GPU. Ces travaux fixent en revanche l’horizon de nos résultats. Notre limite principale — un lot de 256, soit 510 négatifs par ancre, contre plusieurs milliers dans les expériences de référence — est précisément le problème que ces trois familles cherchent à lever, chacune par un mécanisme différent. Nous y revenons en section 8.

## 3 Données

### 3.1 Source et analyse exploratoire

Nous utilisons CIFAR-10 [6], chargé via `torchvision.datasets.CIFAR10` : 60 000 images couleur de  $32 \times 32$  pixels réparties en 10 classes mutuellement exclusives (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). La répartition officielle est de 50 000 images d'entraînement et 10 000 images de test.

Trois propriétés du jeu de données ont guidé nos choix méthodologiques.

- **Équilibre parfait des classes.** Chaque classe compte exactement 6 000 images, dont 5 000 en entraînement et 1 000 en test. L'accuracy est donc une métrique légitime, sans qu'il soit nécessaire de recourir au F1 macro ou à une pondération ; le hasard se situe à 10 %, ce qui donne un point de repère commode.
- **Statistiques des canaux.** Nous avons recalculé les moyennes et écarts-types par canal directement sur les 50 000 images d'entraînement, et ils reproduisent à la quatrième décimale les constantes usuelles : moyennes (0,4914; 0,4822; 0,4465), écarts-types (0,2470; 0,2435; 0,2616). Ces six nombres servent à la normalisation, appliquée à l'identique dans toutes les expériences (`simclr/augmentations.py`). Ce recalcul n'est pas une formalité : reprendre des constantes de normalisation trouvées en ligne sans les vérifier est une source classique d'écart silencieux entre implémentations.
- **Très petite résolution.** Une image CIFAR-10 fait  $32 \times 32$  pixels, contre  $224 \times 224$  pour ImageNet. Cette différence d'un facteur 49 en nombre de pixels a deux conséquences que nous assumons explicitement : elle impose d'adapter la tête d'un ResNet conçu pour ImageNet (section 4.3), et elle rend certaines augmentations inopérantes, voire destructrices — ce que notre ablation confirme pour le flou gaussien (section 5).

### 3.2 Ce que la couleur dit, et ce qu'elle ne dit pas

Une quatrième propriété, moins souvent relevée, explique une partie des résultats de la section 5 et méritait d'être mesurée avant toute expérience.

Nous avons calculé, pour chaque classe, la couleur moyenne de ses images, puis deux dispersions : celle des couleurs moyennes *à l'intérieur* d'une classe, et celle des couleurs moyennes *entre* classes (figure 1). Le résultat est net. L'écart-type intra-classe va de 27,6 (camion) à 39,1 (avion) niveaux, alors que l'écart-type inter-classes ne vaut que 10,4. La couleur varie donc environ trois fois plus d'une image à l'autre *dans* une classe qu'entre les classes elles-mêmes.

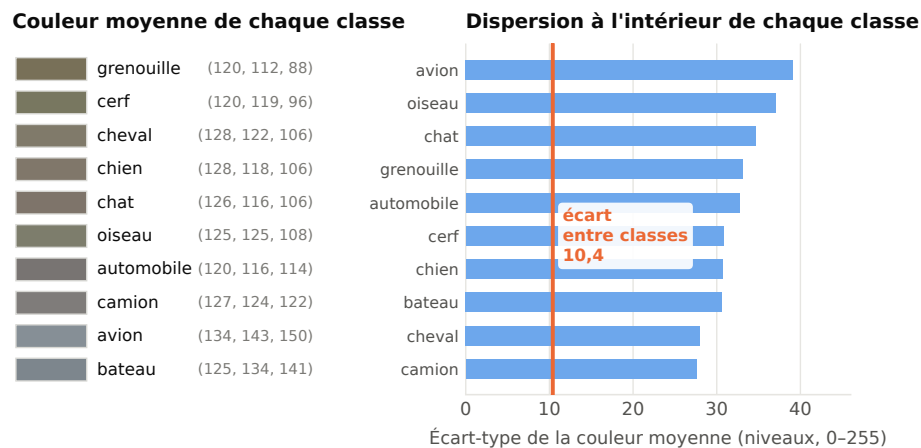
**Conséquence pour la suite.** La couleur d'une image CIFAR-10 est un excellent identifiant *d'instance* et un mauvais indice *de catégorie*. Or la tâche prétexte de SimCLR est précisément une tâche d'identification d'instance : elle demande de reconnaître que deux vues viennent de la même image. La couleur est donc un raccourci disponible, gratuit, et parfaitement suffisant pour résoudre la tâche prétexte sans rien apprendre d'utile pour la classification. La section 5.5 le mesure directement.

### 3.3 Découpage, prévention des fuites

Le jeu de test officiel ne sert qu'une fois, à la toute fin, pour les chiffres publiés : aucun hyperparamètre n'a été choisi en le consultant. Pour disposer néanmoins d'un signal de réglage, nous découpons le train officiel en une réserve étiquetable de 45 000 images et une validation de 5 000 images (figure 2).

Ce découpage obéit à trois règles strictes.

1. Il est **stratifié** : chaque classe est représentée à parts égales dans la validation.
2. Il utilise une **graine dédiée** (7), volontairement distincte des graines d'expérience (42, 123, 2026), et n'est **jamais** refait. Si chaque graine d'expérience voyait une validation différente, l'écart-type que nous rapportons mélangerait deux sources de variation et ne serait plus



La couleur varie 3 fois plus d'une image à l'autre **DANS** une classe qu'entre les classes : elle identifie l'image, pas la catégorie.

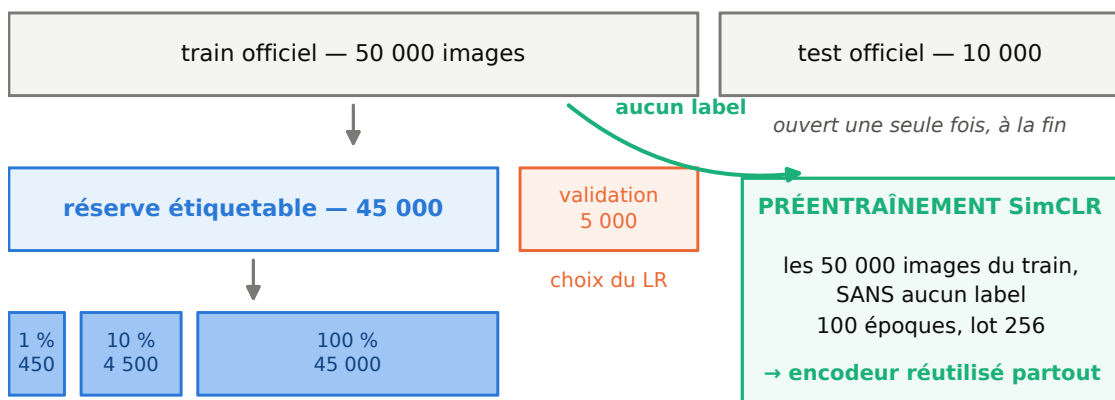
**FIG. 1 :** Analyse exploratoire de la couleur sur les 50 000 images d'entraînement. À gauche, la couleur moyenne de chaque classe : neuf classes sur dix sont des gris-bruns difficiles à distinguer, seuls l'avion et le bateau se détachent par leur dominante bleue. À droite, la dispersion des couleurs moyennes à l'intérieur de chaque classe, comparée à la dispersion entre classes (trait orange).

interprétable.

3. Les trois fractions de labels sont **emboîtées** : les 450 images du régime 1 % sont incluses dans les 4 500 du régime 10 %, elles-mêmes incluses dans les 45 000. Ajouter des labels ne change donc jamais l'identité des labels déjà disponibles.

Le script `scripts/inspecter_split_validation.py` vérifie automatiquement ces trois propriétés (absence de recouvrement train/validation, stratification, emboîtement des fractions) et refuse de valider le protocole si l'une d'elles est violée.

### CIFAR-10 — 60 000 images 32×32, 10 classes



trois fractions stratifiées, emboîtées, 3 graines (42 / 123 / 2026)

**FIG. 2 :** Protocole expérimental. Le pré-entraînement contrastif consomme les 50 000 images du train officiel *sans leurs labels* ; les évaluations aval n'utilisent que la réserve étiquetable, par fractions emboîtées ; le test officiel n'est ouvert qu'une fois.

### 3.4 Choix de périmètre : CIFAR-10 seul

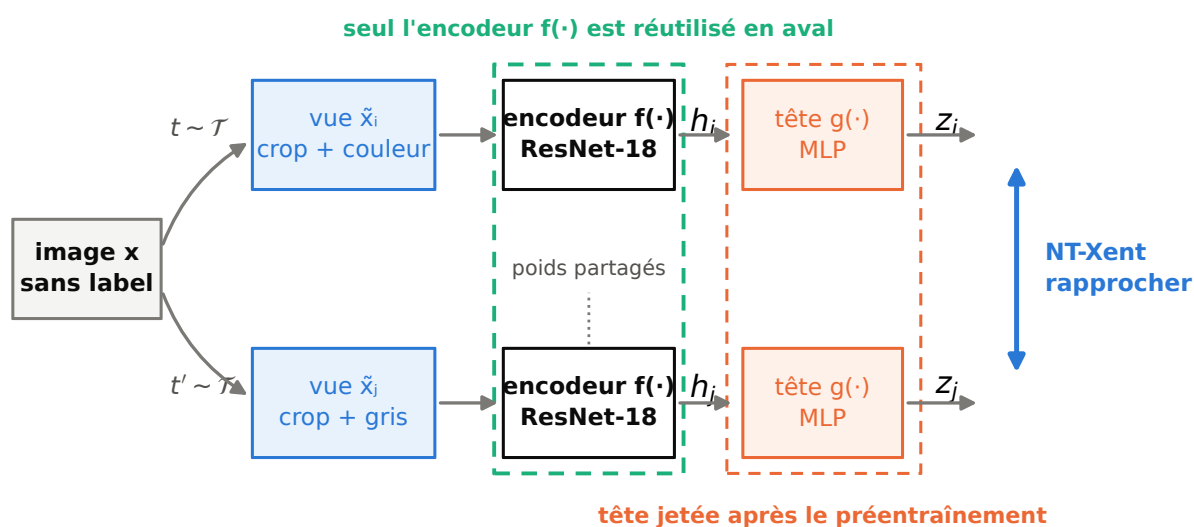
Le sujet cite CIFAR-10 et STL-10 comme jeux de données possibles. Nous avons volontairement restreint le périmètre à CIFAR-10. Aucune des exigences techniques du sujet ne mentionne STL-10, et le budget de calcul disponible (une carte RTX 4060 Laptop,

environ sept heures de GPU au total) était mieux employé à consolider une seule comparaison — trois graines par point, quatre bras, cinq configurations d’ablation — qu’à étaler deux comparaisons plus fragiles sur deux jeux de données. C’est un choix de périmètre assumé, non un oubli ; nous le rappelons en limites (section 8).

## 4 Méthode

### 4.1 Le cadre contrastif

Le principe est illustré figure 3. À partir d’une image  $x$  tirée du jeu non étiqueté, deux transformations  $t, t' \sim \mathcal{T}$  sont tirées indépendamment et produisent deux vues  $\tilde{x}_i = t(x)$  et  $\tilde{x}_j = t'(x)$  : c’est une *paire positive*. Un encodeur partagé  $f(\cdot)$  en extrait les représentations  $h_i = f(\tilde{x}_i)$  et  $h_j = f(\tilde{x}_j)$ , puis une tête de projection  $g(\cdot)$  les envoie vers  $z_i = g(h_i)$  et  $z_j = g(h_j)$ , où la perte contrastive s’applique. Toutes les autres vues du lot constituent les *paires négatives* : aucun échantillonnage explicite de négatifs n’est nécessaire.



**FIG. 3 :** Le cadre SimCLR tel que nous l’implémentons (schéma redessiné d’après [1], figure 2). La perte s’applique sur  $z$  ; c’est pourtant  $h$  qui est réutilisé en aval, la tête de projection étant jetée après le pré-entraînement.

### 4.2 Famille d’augmentations

La famille  $\mathcal{T}$  définit la tâche que le réseau doit résoudre : elle détermine à quelles déformations la représentation devra être invariante. Notre configuration complète enchaîne, dans cet ordre :

- `RandomResizedCrop(32, scale=(0.2, 1.0))` — recadrage aléatoire couvrant de 20 % à 100 % de la surface, puis redimensionnement à  $32 \times 32$ . Jamais retiré : sans lui, les deux vues seraient géométriquement identiques et la tâche contrastive perdrait son sens.
- `RandomHorizontalFlip(p=0.5)` — retournement horizontal.
- `ColorJitter(0.4, 0.4, 0.4, 0.1)` appliqué avec  $p = 0,8$  — perturbation de luminosité, contraste, saturation et teinte.
- `RandomGrayscale(p=0.2)` — conversion en niveaux de gris.
- `GaussianBlur(kernel=3, sigma=(0.1, 1.0))` appliqué avec  $p = 0,5$  — désactivé dans la configuration principale, activé dans une variante d’ablation.
- Conversion en tenseur et normalisation par les statistiques de la section 3.

Chaque brique est pilotable indépendamment, ce qui rend l’ablation de la section 5 possible

sans dupliquer le code.

### 4.3 Encodeur

Nous utilisons un ResNet-18 [5], adapté à la petite résolution de CIFAR-10 par deux modifications standard :

- la première convolution passe d'un noyau  $7 \times 7$  de pas 2 à un noyau  $3 \times 3$  de pas 1. Sur une image de 32 pixels de côté, un noyau  $7 \times 7$  de pas 2 réduit immédiatement la carte à  $16 \times 16$  et efface les détails fins ;
- la couche de *max pooling* qui suit est remplacée par l'identité, pour la même raison : elle diviserait encore la résolution par deux avant tout traitement utile.

La couche entièrement connectée finale, qui produirait 1 000 scores ImageNet, est également remplacée par l'identité : la sortie de l'encodeur est la représentation  $h \in \mathbb{R}^{512}$  qui la précède. L'ensemble représente environ 11,2 millions de paramètres.

### 4.4 Tête de projection

$g(\cdot)$  est un perceptron à une couche cachée :  $512 \rightarrow 512 \rightarrow 128$ , avec une activation ReLU intermédiaire. La perte s'applique donc sur  $z \in \mathbb{R}^{128}$ . La tête est **jetée** après le pré-entraînement : toutes les évaluations aval n'utilisent que  $f(\cdot)$  et la représentation  $h$ .

### 4.5 Perte NT-Xent

Pour un lot de  $N$  images, les  $2N$  vues sont normalisées sur la sphère unité, puis toutes les similarités cosinus sont calculées. Pour une ancre  $i$  dont le positif est  $j$  :

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)} \quad \text{avec} \quad \text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}. \quad (1)$$

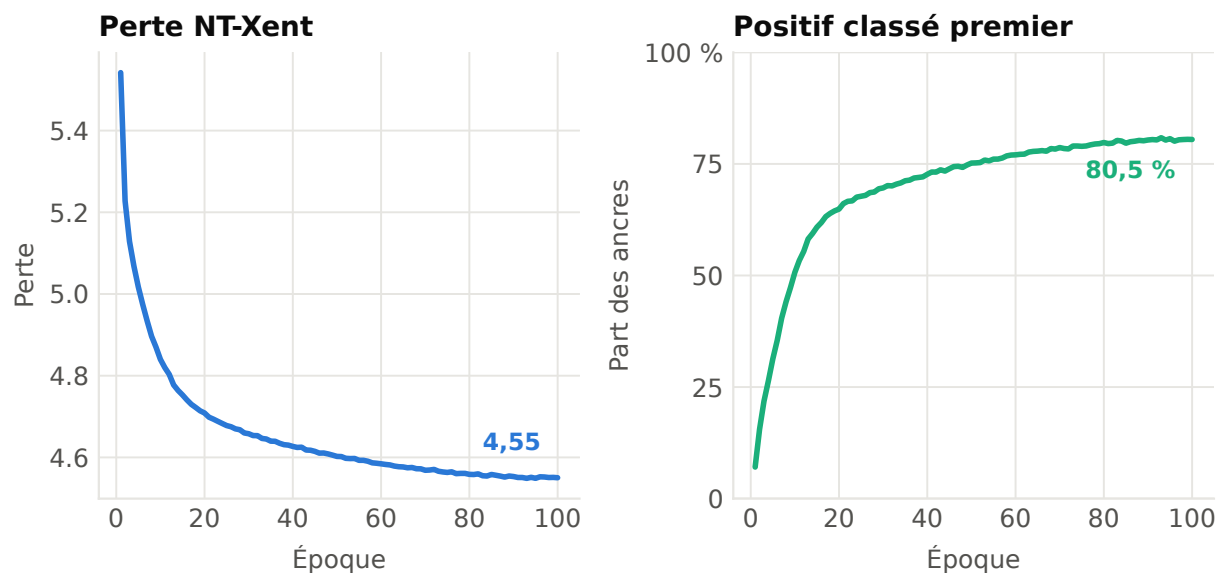
Le dénominateur parcourt les  $2N - 1$  autres vues du lot : à lot 256, cela fait 510 négatifs par ancre. La température  $\tau = 0,5$  contrôle la dureté de la discrimination : une température basse force une séparation nette entre positifs et négatifs, une température élevée lisse la distribution des similarités.

Notre implémentation (`simclr/contrastif.py`) exploite le fait que l'équation (1) est exactement une entropie croisée : après suppression de la diagonale de la matrice de similarités, chaque ligne devient un vecteur de  $2N - 1$  scores dont la cible est la position du positif, et `F.cross_entropy` calcule directement  $\mathcal{L}$ . Ce choix évite une boucle explicite et rend le code vérifiable ligne à ligne.

### 4.6 Pré-entraînement

Le pré-entraînement voit les 50 000 images du train officiel, *sans jamais lire un seul label*. Hyperparamètres : 100 époques, lot 256, optimiseur AdamW avec taux d'apprentissage  $3 \cdot 10^{-4}$  et *weight decay*  $10^{-4}$ , échauffement linéaire sur 10 époques suivi d'une décroissance en cosinus jusqu'à  $10^{-6}$ , température 0,5, précision mixte automatique, graine 42. Un unique pré-entraînement est réalisé par configuration d'augmentations, puis réutilisé pour tous les régimes de labels : c'est le poste de calcul dominant et le recalculer à chaque point de la courbe serait un gaspillage.

La durée médiane est de 30,4 s par époque sur une RTX 4060 Laptop, soit environ 51 minutes pour 100 époques, avec un pic mémoire de 1,37 Go. Quelques époques isolées atteignent plusieurs minutes du fait de la contention de la machine ; nous en tirons la conséquence en section 8 et ne rapportons *aucune* comparaison de coût en temps entre méthodes.



**Fig. 4 :** Pré-entraînement contrastif, 100 époques. À gauche la perte NT-Xent, à droite la part des ancres dont le positif est classé au premier rang parmi les 511 candidats. Les deux courbes sont des diagnostics d'optimisation : elles indiquent que l'entraînement se déroule normalement, mais *ne mesurent pas* la qualité de la représentation — la section 5.4 montre qu'elles peuvent même la contredire.

#### 4.7 Trois protocoles d'évaluation, plus un témoin

Un encodeur pré-entraîné peut être exploité de deux manières, et le sujet demande de comparer avec et sans pré-entraînement. Nous évaluons donc quatre bras.

1. **Évaluation linéaire** (*linear probe*). L'encodeur est gelé; seule une couche Linear(512, 10) est entraînée sur les représentations  $h$  standardisées. Environ 5 130 paramètres entraînables. C'est le protocole standard pour comparer la qualité *intrinsèque* de deux pré-entraînements, et c'est celui que nous utilisons dans l'ablation.
2. **Fine-tuning complet**. L'encodeur pré-entraîné et la tête de classification s'adaptent ensemble. Environ 11,2 millions de paramètres entraînables. C'est la performance réellement utile en pratique.
3. **Supervisé from scratch**. Strictement le même entraînement que le fine-tuning, à partir de poids aléatoires. C'est la baseline concurrente.
4. **Témoin à encodeur aléatoire gelé**. Le protocole d'évaluation linéaire appliqué à un ResNet-18 jamais entraîné. Ce bras n'est demandé nulle part; nous l'ajoutons parce qu'il est le seul moyen de distinguer ce que le pré-entraînement apporte de ce que l'architecture et le protocole d'évaluation apportent à eux seuls.

#### 4.8 Égalité des budgets de calcul

Le piège le plus courant dans ce type de comparaison consiste à opposer deux méthodes qui n'ont pas reçu le même budget d'entraînement. Le tableau 1 liste les hyperparamètres des deux bras comparés, extraits des fichiers `resultats.json` produits par les runs eux-mêmes, dans `outputs/finetuning_simclr_45k/` et `outputs/supervise_from_scratch_45k/`.

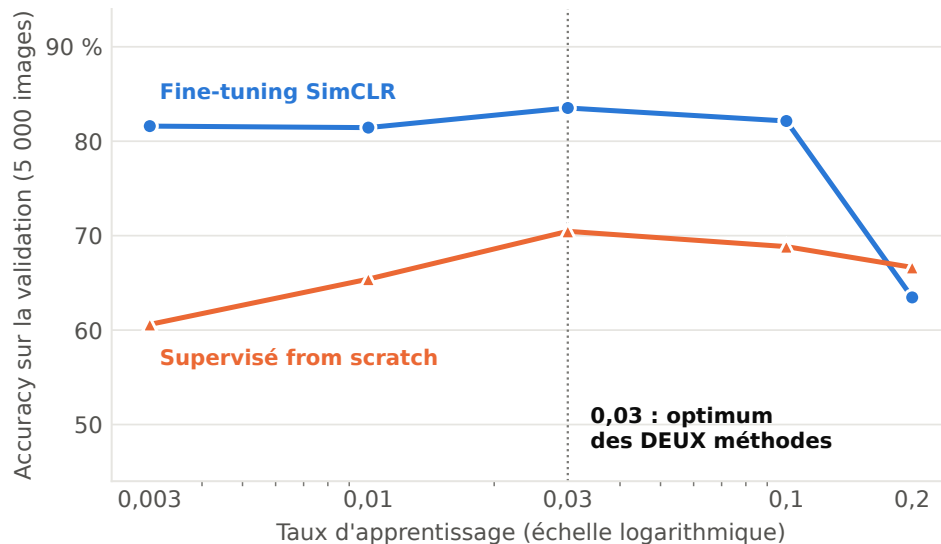
#### 4.9 Choix du taux d'apprentissage

Le taux d'apprentissage est le seul hyperparamètre que nous avons réglé, et nous l'avons réglé *symétriquement* : la même grille  $\{0,2; 0,1; 0,03; 0,01; 0,003\}$  a été balayée pour les deux méthodes, sur la validation, au régime 10 % des labels (figure 5). Aucune des deux ne bénéficie donc d'un avantage de réglage.

**TAB. 1 :** Hyperparamètres des deux bras comparés. La seule différence entre les deux colonnes est l’initialisation des poids de l’encodeur.

|                              | Fine-tuning SimCLR                    | Supervisé <i>from scratch</i>         |
|------------------------------|---------------------------------------|---------------------------------------|
| Initialisation de l’encodeur | poids pré-entraînés (100 époques SSL) | poids aléatoires                      |
| Architecture                 | ResNet-18 adapté, ~11,2 M             | ResNet-18 adapté, ~11,2 M             |
| Époques                      | 100                                   | 100                                   |
| Taille de lot                | 256                                   | 256                                   |
| Optimiseur                   | SGD, momentum 0,9, Nesterov           | SGD, momentum 0,9, Nesterov           |
| Taux d’apprentissage         | 0,03 (cosinus $\rightarrow 10^{-4}$ ) | 0,03 (cosinus $\rightarrow 10^{-4}$ ) |
| <i>Weight decay</i>          | $5 \cdot 10^{-4}$                     | $5 \cdot 10^{-4}$                     |
| Augmentations d’entraînement | RandomCrop(32, pad=4) + Flip          | RandomCrop(32, pad=4) + Flip          |
| Précision mixte              | activée                               | activée                               |
| Découpage / graines          | seed_split 7; 42, 123, 2026           | seed_split 7; 42, 123, 2026           |

Les deux courbes atteignent leur maximum en 0,03, valeur retenue pour les deux bras. La valeur 0,2 dégrade fortement les deux méthodes, ce qui indique que la grille encadre bien l’optimum plutôt que de s’arrêter à sa frontière.



**FIG. 5 :** Balayage symétrique du taux d’apprentissage sur la validation, au régime 10% des labels. Même grille pour les deux méthodes.

#### 4.10 Métriques, graines et reproductibilité

La métrique publiée est l’accuracy top-1 sur les 10 000 images du test officiel. Chaque cellule des tableaux est la moyenne de trois répétitions (graines 42, 123, 2026), accompagnée de l’écart-type. Les graines contrôlent l’initialisation, l’ordre des lots et le tirage des sous-ensembles étiquetés ; le découpage train/validation, lui, reste fixe. Chaque ligne de résultat est reproductible par une commande unique, documentée dans docs/REPRODUCTION.md du dépôt.

## 5 Étude d’ablation : quelles augmentations comptent ?

### 5.1 Question et protocole

Le sujet demande d’étudier l’effet du choix des augmentations sur la qualité des représentations apprises. Nous adoptons un design **additif** : on part du recadrage seul et on ajoute *une* brique à la fois. C’est la règle d’or d’une ablation — une seule variable change

entre deux lignes, tout le reste est identique.

Les cinq configurations partagent rigoureusement : ResNet-18 adapté, 100 époques, lot 256, AdamW  $3 \cdot 10^{-4}$ , échauffement 10 époques puis cosinus, température 0,5, précision mixte, graine de pré-entraînement 42. L'évaluation est un *linear probe* sur encodeur gelé, moyenné sur trois graines.

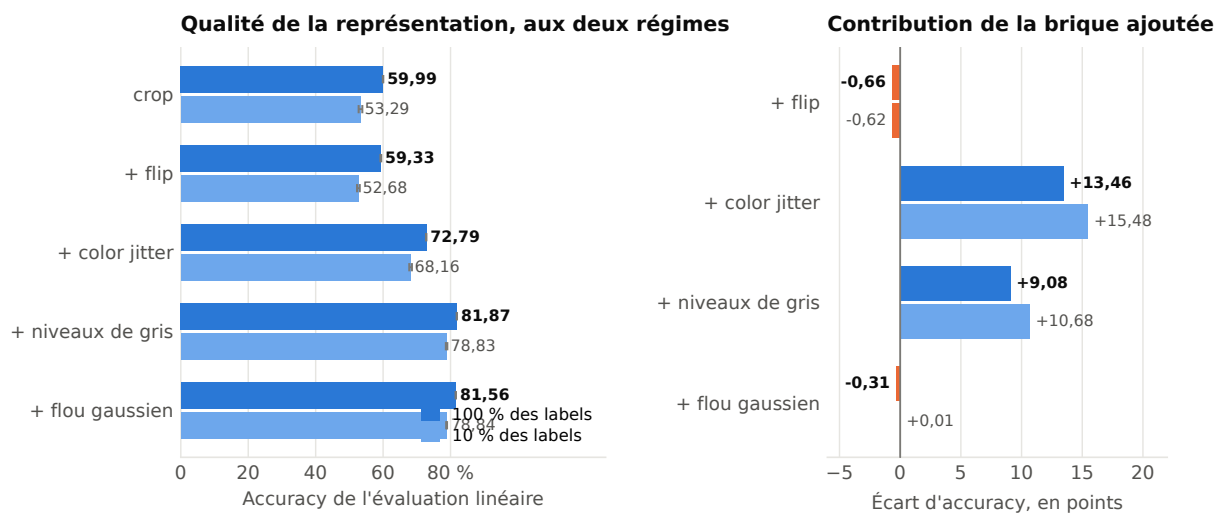
Nous rapportons cette évaluation à **deux** régimes d'annotation. À 10 % des labels d'abord, parce que c'est le protocole le plus discriminant : avec peu d'étiquettes, le classifieur ne peut pas compenser une représentation médiocre, et la qualité de l'encodeur pèse donc au maximum. À 100 % ensuite, parce que la mesure y est nettement moins bruitée — l'écart-type de la configuration la plus instable passe de  $\pm 0,54$  à  $\pm 0,06$ . Les deux commandes sont python `-m scripts.reproduire_ablation` et la même avec l'option `--fraction-probe 0.10`; ensemble, elles régénèrent l'intégralité du tableau 2.

## 5.2 Résultats

**TAB. 2 :** Ablation additive des augmentations. Les deux colonnes de gauche sont des *diagnostics* de la tâche prétexte et ne servent jamais au classement. Les deux colonnes de droite donnent l'accuracy de l'évaluation linéaire aux deux régimes d'annotation ; le classement est identique dans les deux. Écart-type sur trois graines de *probe*.

| Configuration                    | <i>diagnostics de la tâche prétexte</i> |                         | accuracy de l'évaluation linéaire  |                                    |
|----------------------------------|-----------------------------------------|-------------------------|------------------------------------|------------------------------------|
|                                  | Perte NT-Xent                           | Positif 1 <sup>er</sup> | à 10 % des labels                  | à 100 % des labels                 |
| crop seul                        | 4,3739                                  | 98,01 %                 | $53,29 \pm 0,54$                   | $59,99 \pm 0,06$                   |
| + flip                           | 4,3802                                  | 97,95 %                 | $52,68 \pm 0,40$                   | $59,33 \pm 0,11$                   |
| + color jitter                   | 4,4674                                  | 91,68 %                 | $68,16 \pm 0,39$                   | $72,79 \pm 0,04$                   |
| + niveaux de gris <sup>(a)</sup> | 4,5502                                  | 80,47 %                 | <b><math>78,83 \pm 0,22</math></b> | <b><math>81,87 \pm 0,04</math></b> |
| + flou gaussien                  | 4,5601                                  | 78,86 %                 | $78,84 \pm 0,13$                   | $81,56 \pm 0,03$                   |

<sup>(a)</sup> Configuration retenue pour toutes les expériences de la section 6.



les deux briques de couleur : +22,54 points à 100 %, +26,16 à 10 %

**FIG. 6 :** Ablation des augmentations. À gauche, le niveau atteint par chaque configuration ; à droite, la contribution propre de la brique ajoutée.

### 5.3 Discussion, variante par variante

La figure 6 décompose ces chiffres : à gauche le niveau atteint par chaque configuration, à droite la contribution propre de la brique ajoutée. Nous commentons chaque variante ci-dessous.

**Les deux régimes donnent le même classement.** C'est le premier point à relever : l'ordre des cinq configurations est rigoureusement identique à 10 % et à 100 % des labels. La conclusion de l'ablation ne dépend donc pas du régime d'annotation choisi pour l'évaluer, ce qui la rend nettement plus robuste qu'une mesure à un seul point.

Les *amplitudes*, en revanche, diffèrent, et dans un sens instructif. À 10 % des labels, les deux briques de couleur apportent +26,16 points (+15,48 pour le color jitter, +10,68 pour les niveaux de gris) contre +22,54 à 100 %. Autrement dit : plus les étiquettes sont rares, plus la qualité de la représentation compte. C'est cohérent avec le résultat central du projet — quand les labels abondent, le classifieur peut compenser une représentation médiocre ; quand ils manquent, il ne le peut plus.

**Le retournement horizontal :** -0,66 point à 100 %, -0,62 à 10 %. Aucun effet mesurable. Le retournement ne modifie ni les couleurs ni le contenu de l'image ; il n'ajoute donc aucune difficulté à la tâche contrastive. L'écart observé est du même ordre que le bruit d'évaluation et nous ne le rapportons *pas* comme un effet négatif.

**La distorsion de couleur :** +13,46 points à 100 %, +15,48 à 10 %. C'est le gain le plus important de l'étude, et de loin. Il retrouve indépendamment le résultat central de [1] : sans perturbation des couleurs, les deux vues d'une même image conservent des histogrammes de couleur quasi identiques, ce qui offre au réseau un raccourci lui permettant de résoudre la tâche sans jamais analyser la forme.

**Les niveaux de gris :** +9,08 points à 100 %, +10,68 à 10 %. Second gain majeur. Cette brique supprime le résidu du raccourci précédent : lorsqu'une des deux vues est convertie en niveaux de gris, la comparaison par teinte devient impossible et la forme redevient le seul signal exploitable. Cumulées, les deux briques de couleur apportent +22,54 points — soit la quasi-totalité de l'écart entre la pire et la meilleure configuration.

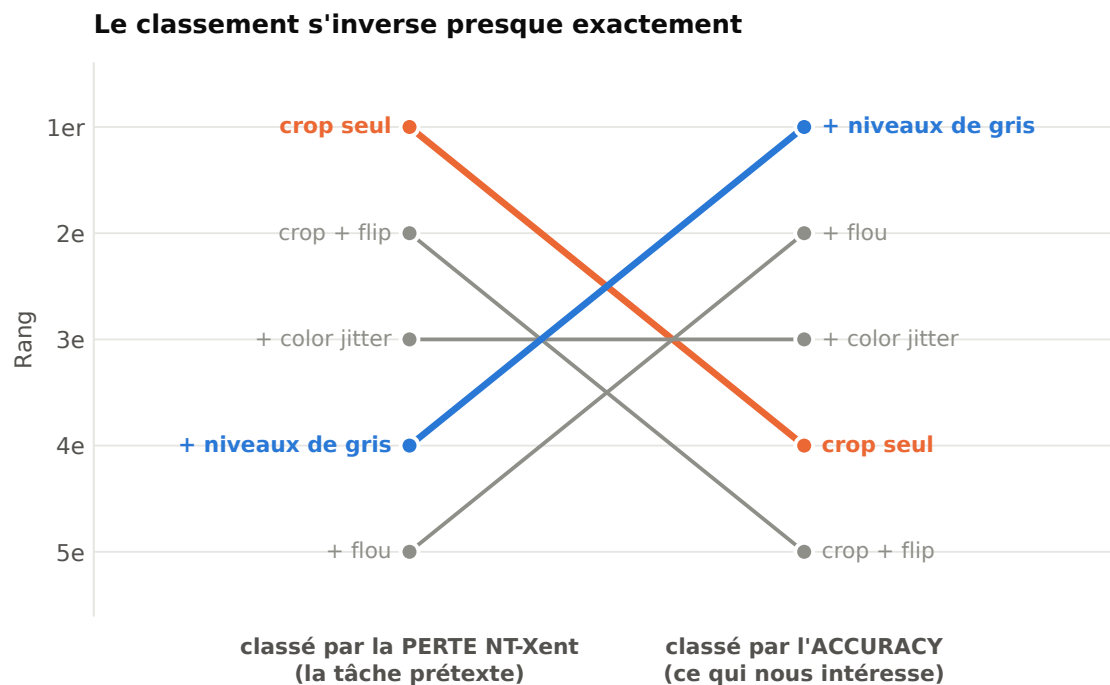
**Le flou gaussien :** -0,31 point à 100 %, +0,01 à 10 %. Aucun effet mesurable non plus — et le fait que le signe s'inverse entre les deux régimes, pour une amplitude aussi faible, confirme qu'il s'agit de bruit et non d'un effet. Ce résultat diffère de celui rapporté sur ImageNet, où le flou aide. La divergence s'explique par la résolution et non par une contradiction : sur une image de  $224 \times 224$  pixels, un noyau de flou reste local ; sur une image de  $32 \times 32$ , un noyau  $3 \times 3$  couvre presque 10 % de la largeur de l'image et détruit une part significative de l'information disponible. Nous en concluons que le flou n'est pas transférable tel quel à cette échelle, non qu'il serait inutile en général.

**Réponse à la question du sujet.** Sur CIFAR-10, ce sont les augmentations de *couleur* qui portent la qualité de la représentation : +22,54 points à 100 % des labels et +26,16 points à 10 % pour la distorsion de couleur et les niveaux de gris réunis, contre des écarts non mesurables pour le retournement horizontal et le flou gaussien. Le classement est identique aux deux régimes.

### 5.4 Bien résoudre la tâche prétexte n'est pas bien apprendre

Le résultat le plus instructif de cette ablation n'est pas le classement lui-même, mais sa comparaison avec le classement par la perte.

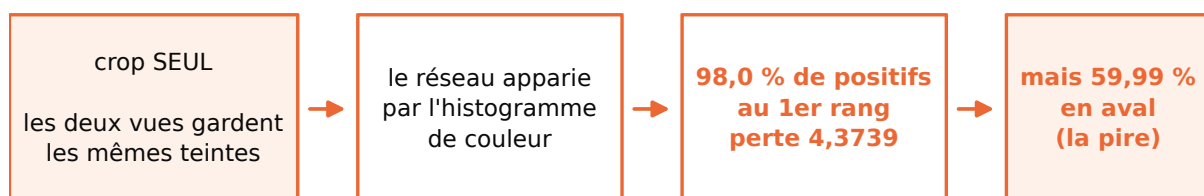
La configuration *crop seul* obtient *la meilleure* perte NT-Xent finale (4,3739) et place 98,01 % des positifs au premier rang. Elle résout donc presque parfaitement la tâche prétexte — et produit la *pire* représentation de l'étude (59,99 %). Symétriquement, la configuration complète obtient une perte plus élevée (4,5502), un taux de positifs au premier rang bien plus faible (80,47 %), et la meilleure représentation (81,87 %). La figure 7 montre que le classement s'inverse presque exactement d'un critère à l'autre.



**FIG. 7 :** Rang de chaque configuration selon la perte NT-Xent (à gauche) et selon l'accuracy de l'évaluation linéaire (à droite).

Le mécanisme est identifiable, et c'est ce qui rend l'observation solide (figure 8) : retirer les perturbations de couleur laisse aux deux vues le même histogramme de couleur, ce qui permet de les apparier sans analyser la forme. La tâche devient facile, la perte descend — et la représentation apprise se réduit à un descripteur de couleur, inutile en aval.

### Pourquoi « crop seul » résout la tâche sans rien apprendre



Perturber les couleurs supprime le raccourci : la tâche devient plus dure (perte 4,5502, 80,5 % au 1er rang) et la représentation passe à 81,87 %.

**FIG. 8 :** Le raccourci de couleur, mécanisme derrière l'inversion.

**À retenir.** Une perte contrastive plus basse peut signaler une tâche prétexte trop facile, donc une représentation plus pauvre. Le seul critère de classement valide est la performance sur la tâche aval. C’est pourquoi la commande de reproduction de l’ablation affiche explicitement les deux colonnes de diagnostic comme *non décisionnelles*.

Nous nous gardons de transformer cette observation en loi générale. La corrélation de rang entre perte et accuracy vaut  $\rho = +0,80$ , mais sur  $n = 5$  configurations seulement, ce qui ne permet aucune conclusion statistique ( $p \approx 0,10$ ). Ce que nous affirmons est plus modeste et mieux étayé : sur nos cinq configurations, les deux classements s’opposent, et un mécanisme concret explique pourquoi.

### 5.5 Le raccourci, mesuré sans aucun réseau de neurones

L’explication précédente reste une interprétation tant qu’elle n’est pas mesurée. Nous avons donc conçu un diagnostic qui la teste directement, et dont la particularité est de **n’employer aucun réseau de neurones** : il ne mesure pas ce que notre modèle a appris, mais ce que les données et les augmentations rendent possible.

**Protocole.** Sur 2 000 images d’entraînement, nous produisons deux vues par image avec la chaîne d’augmentations étudiée. Chaque vue est résumée par un histogramme de couleur de  $3 \times 8 = 24$  bins, normalisé — une description qui ignore complètement la forme, la texture et la position. Nous mesurons ensuite deux quantités : le taux de vues dont le véritable positif est le plus proche voisin au sens de la distance  $L^1$  entre histogrammes, parmi les 2 000 candidates ; et le rapport entre la distance moyenne aux négatifs et la distance moyenne au positif. Le hasard se situe à  $1/2000 = 0,05 \%$ .

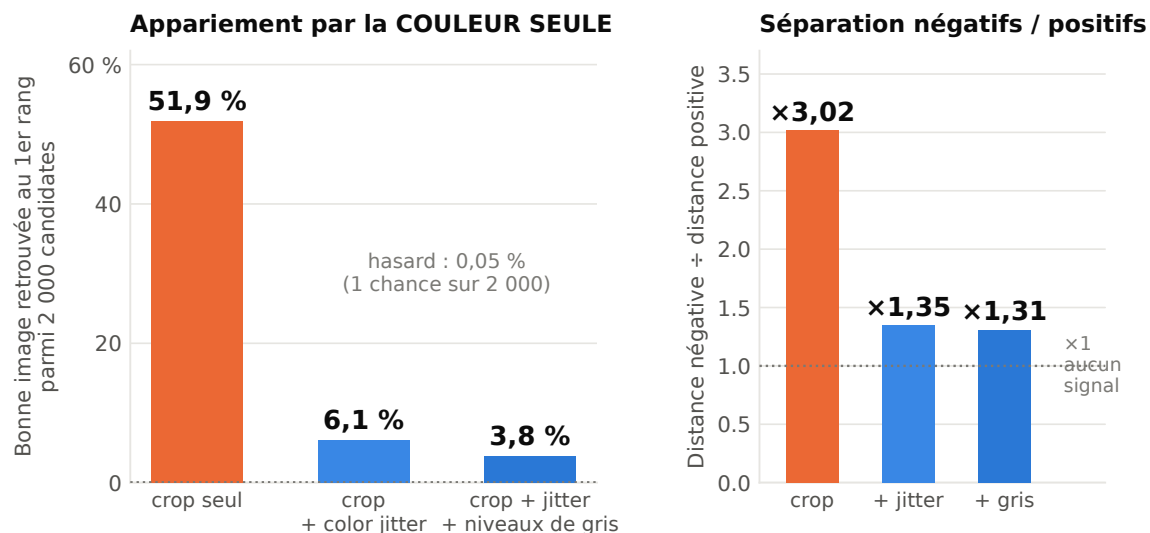
**TAB. 3 :** Diagnostic du raccourci de couleur, mené sur les données seules. Un simple histogramme de couleur suffit-il à résoudre la tâche prétexte ?

| Augmentations appliquées | Positif retrouvé | Distance nég. / pos. | Accuracy de l’encodeur |
|--------------------------|------------------|----------------------|------------------------|
| crop seul                | <b>51,9 %</b>    | $\times 3,02$        | 59,99 %                |
| + color jitter           | 6,1 %            | $\times 1,35$        | 72,79 %                |
| + niveaux de gris        | 3,8 %            | $\times 1,31$        | <b>81,87 %</b>         |
| <i>hasard</i>            | 0,05 %           | $\times 1,00$        | 10 %                   |

**Résultat.** Avec le recadrage seul, un histogramme de couleur retrouve la bonne image dans 51,9 % des cas parmi 2 000 candidates, soit plus de mille fois le hasard. La tâche prétexte est donc, littéralement, à moitié résolue avant même qu’un réseau ne regarde une seule forme. Ajouter la distorsion de couleur fait tomber ce taux à 6,1 %, et les niveaux de gris à 3,8 % ; le rapport de distances passe de  $\times 3,02$  à  $\times 1,31$ , c’est-à-dire presque plus rien.

**Interprétation.** Ce diagnostic transforme notre explication en mesure, et il éclaire les trois colonnes du tableau 2 d’un seul coup. Les 98,01 % de positifs au premier rang obtenus par la configuration *crop seul* ne témoignent pas d’une représentation supérieure : plus de la moitié de ce score était accessible à un histogramme de couleur, sans le moindre apprentissage. Inversement, la chute du taux de positifs à 80,47 % pour la configuration complète n’est pas une dégradation — c’est le signe que le raccourci a été fermé et que le réseau a dû apprendre autre chose. Le gain de +22,54 points en aval est le prix de cette fermeture.

**Ce que nous pouvons affirmer.** Le raccourci de couleur n’est pas une hypothèse commode pour expliquer nos chiffres : il est mesurable sur les données brutes, il est massif (51,9 % contre 0,05 % au hasard), et les deux augmentations qui le neutralisent sont exactement celles qui apportent les +22,54 points de notre ablation.



Diagnostic mené sur les données seules : histogramme  $3 \times 8$  bins, distance L1, aucun réseau de neurones n'intervient.

**Fig. 9 :** Le raccourci de couleur, mesuré directement sur les données. À gauche, la part des vues dont le vrai positif est retrouvé au premier rang par la seule couleur, parmi 2 000 candidates. À droite, le rapport entre distance aux négatifs et distance au positif : au-dessus de  $\times 1$ , la couleur porte un signal exploitable.

**Limites de ce diagnostic.** Il mesure une *borne inférieure* de ce que la couleur permet, pas une borne supérieure de ce que le réseau exploite : un descripteur de couleur plus fin (histogrammes spatiaux, moments d'ordre supérieur) ferait probablement mieux. Il porte sur 2 000 images d'un seul lot de données, avec une graine unique. Enfin, notre implémentation des transformations de couleur est une reproduction en numpy de ColorJitter et RandomGrayscale ; elle en respecte les paramètres et l'ordre d'application, mais n'est pas le code de torchvision lui-même.

## 5.6 Ce que cette ablation ne permet pas d'affirmer

Trois limites doivent accompagner le tableau 2, et nous préférons les énoncer plutôt que d'attendre qu'elles nous soient opposées.

- Les  $\pm 0,0X$  sont l'écart-type sur les **trois graines du linear probe uniquement**. Le pré-entraînement n'a tourné qu'**une seule fois** par configuration (graine 42) : la variabilité d'un pré-entraînement à l'autre est inconnue. Trois graines de pré-entraînement sur cinq configurations représenteraient environ dix heures de GPU supplémentaires, hors de notre budget.
- Par conséquent, +13,46 et +9,08 sont des effets trop grands pour être du bruit et nous les traitons comme des conclusions solides ; -0,66 et -0,31 sont rapportés comme « aucun effet mesurable » et jamais comme des effets négatifs.
- L'ablation est additive, non factorielle : elle mesure l'apport de chaque brique *dans l'ordre où nous les ajoutons*. Elle ne dit rien des interactions entre briques prises dans un autre ordre — par exemple ce que donnerait « crop + niveaux de gris » sans distorsion de couleur.

## 6 Résultats

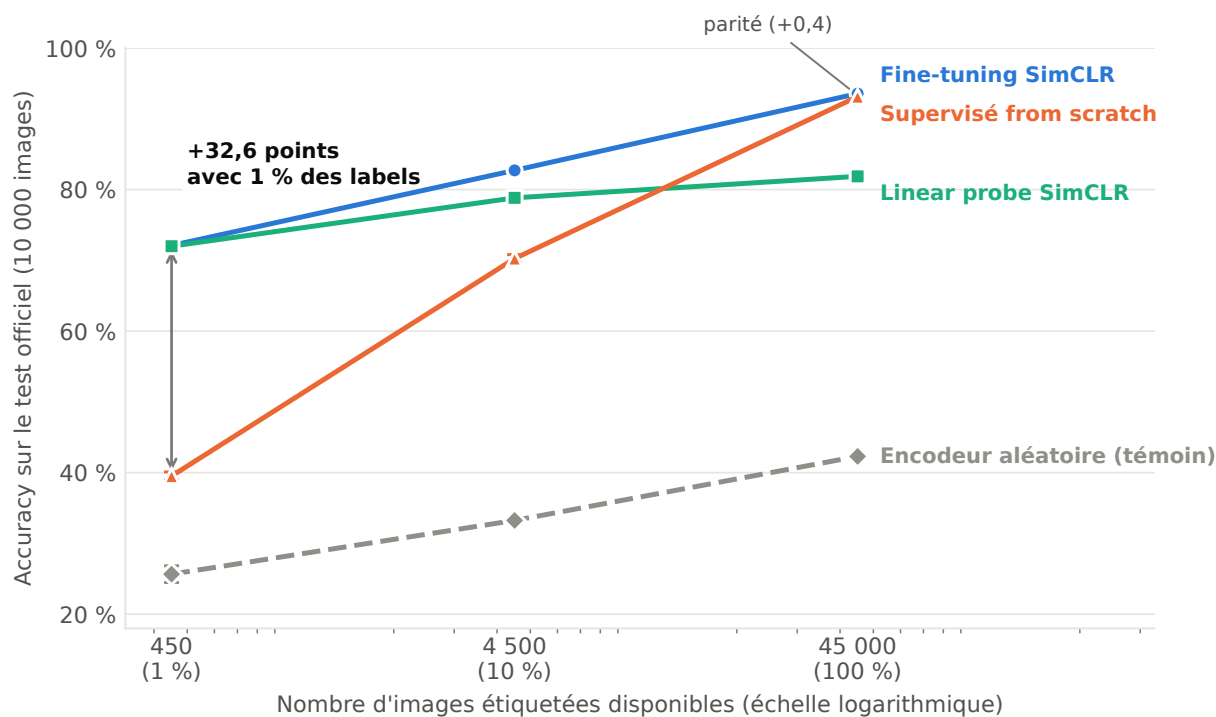
### 6.1 Tableau principal

Le tableau 4 et la figure 10 constituent le résultat central du projet.

**TAB. 4 :** Accuracy top-1 sur le test officiel (10 000 images), en %, moyenne  $\pm$  écart-type sur trois graines. Toutes les méthodes sont appariées : même découpage, mêmes graines, mêmes sous-ensembles étiquetés.

| Méthode                                | Poids entraînés      | 450 (1 %)                          | 4 500 (10 %)                       | 45 000 (100 %)                     |
|----------------------------------------|----------------------|------------------------------------|------------------------------------|------------------------------------|
| Encodeur aléatoire gelé <sup>(t)</sup> | $\sim 5\,130$        | $25,66 \pm 1,18$                   | $33,25 \pm 0,46$                   | $42,30 \pm 0,02$                   |
| Linear probe SimCLR                    | $\sim 5\,130$        | $72,00 \pm 0,40$                   | $78,83 \pm 0,22$                   | $81,87 \pm 0,04$                   |
| Fine-tuning SimCLR                     | $\sim 11,2\text{ M}$ | <b><math>72,10 \pm 0,86</math></b> | <b><math>82,71 \pm 0,35</math></b> | <b><math>93,56 \pm 0,09</math></b> |
| Supervisé <i>from scratch</i>          | $\sim 11,2\text{ M}$ | $39,52 \pm 0,58$                   | $70,25 \pm 0,56$                   | $93,13 \pm 0,08$                   |
| Écart fine-tuning – supervisé          |                      | +32,58                             | +12,46                             | +0,43                              |

<sup>(t)</sup> Témoin diagnostique : même architecture et même protocole d'évaluation, poids jamais entraînés.



**FIG. 10 :** Accuracy de test en fonction du nombre d'images étiquetées, avec et sans pré-entraînement auto-supervisé. Les barres d'erreur représentent l'écart-type sur trois graines ; elles sont souvent plus petites que les marqueurs.

## 6.2 Lecture des trois régimes

**450 labels : le pré-entraînement change la nature du problème.** L'écart de +32,58 points est le résultat le plus net de l'étude. Avec 45 images étiquetées par classe, la baseline supervisée atteint 39,52 %; le même réseau, initialisé par le pré-entraînement contrastif, atteint 72,10 %. Formulé du point de vue du coût d'annotation : le modèle pré-entraîné atteint 72,1 % avec 450 labels, tandis que la baseline a besoin de 4 500 labels — dix fois plus — pour n'atteindre que 70,25 %. L'économie d'annotation dépasse 90 % à performance équivalente.

**450 labels, second constat : dégeler ne sert à rien.** À ce régime, le fine-tuning complet (72,10 %) et l'évaluation linéaire sur encodeur gelé (72,00 %) sont séparés de 0,10 point, soit nettement moins que l'écart-type du premier ( $\pm 0,86$ ). Autrement dit : avec très peu de labels, entraîner 11,2 millions de paramètres n'apporte rien de mesurable par rapport à en entraîner 5 130. Le gradient disponible est trop maigre pour améliorer l'encodeur, et le geler évite qu'il ne se dégrade. C'est un résultat directement actionnable en pratique : à faible supervision, l'évaluation linéaire est aussi bonne et deux mille fois moins coûteuse en paramètres.

**4 500 labels : le régime intermédiaire.** L'écart se réduit à +12,46 points. Le fine-tuning dépasse ici l'évaluation linéaire (82,71 contre 78,83) : il y a désormais assez de signal pour que l'adaptation de l'encodeur devienne rentable.

**45 000 labels : parité.** Avec toutes les annotations, les deux méthodes se rejoignent (93,56 contre 93,13). Le bénéfice du SSL s'annule quand les labels abondent, exactement comme l'anticipe la littérature. Nous insistons sur un point : nous parlons de **parité** et nous ne revendiquons *pas* l'écart de +0,43 point comme une supériorité — voir section 8.

**Le témoin.** Sans lui, un lecteur sceptique pourrait attribuer les 32 points au protocole d'évaluation plutôt qu'au pré-entraînement. Le témoin ferme cette porte : à 450 labels, un encodeur aléatoire gelé évalué exactement comme le nôtre atteint 25,66 %, contre 72,00 % pour l'encodeur pré-entraîné — 46,34 points d'écart, à architecture, protocole et nombre de paramètres entraînaibles strictement identiques. La seule variable qui change est le contenu des poids de l'encodeur.

Le témoin apporte un second enseignement, moins attendu. Un encodeur *jamais entraîné* atteint tout de même 42,30 % avec 45 000 labels, très au-dessus des 10 % du hasard. Une convolution aléatoire n'est donc pas un bruit : la structure convolutionnelle elle-même — localité, partage de poids, invariance par translation — extrait déjà des descripteurs partiellement linéairement séparables. C'est un rappel utile pour lire les comparaisons de la littérature : la bonne référence n'est jamais le hasard à 10 %, mais ce que la même architecture obtient sans apprentissage.

## 6.3 Lecture en coût d'annotation

Les points d'accuracy sont la métrique naturelle du chercheur ; le nombre d'annotations est la métrique naturelle de celui qui finance le projet. La courbe se lit aussi horizontalement.

**TAB. 5 :** Lecture horizontale de la figure 10 : combien d'images faut-il annoter pour atteindre un niveau donné ?

| Niveau visé | Avec pré-entraînement  | Sans pré-entraînement  | Économie |
|-------------|------------------------|------------------------|----------|
| ≈ 70 %      | 450 images (72,10 %)   | 4 500 images (70,25 %) | ×10      |
| ≈ 82 %      | 4 500 images (82,71 %) | entre 4 500 et 45 000  | > ×3     |
| ≈ 93 %      | 45 000 images          | 45 000 images          | aucune   |

La première ligne est celle qui porte l’argument économique : atteindre 70 % demande 450 annotations avec pré-entraînement contre 4 500 sans, soit une réduction de 90 % du volume à annoter. Si l’on transpose à un contexte où chaque annotation demande l’avis d’un spécialiste, c’est la différence entre une demi-journée de travail et une semaine.

La deuxième ligne est encadrée plutôt que chiffrée, et il faut le dire : nous n’avons mesuré que trois points, nous ne pouvons donc pas interpoler le nombre exact d’annotations que la baseline supervisée exigerait pour atteindre 82 %. Nous savons seulement qu’il est supérieur à 4 500 (elle y plafonne à 70,25 %) et inférieur à 45 000 (elle y atteint 93,13 %). Une mesure à 20 000 labels lèverait cette incertitude ; elle n’était pas dans notre budget.

#### 6.4 Le régime de faible supervision, dans le détail

Un point mérite d’être isolé car il est directement actionnable et n’apparaît dans aucun des documents de référence du sujet.

À 450 labels, le fine-tuning complet ( $72,10 \pm 0,86$ ) et l’évaluation linéaire sur encodeur gelé ( $72,00 \pm 0,40$ ) sont séparés de 0,10 point, soit huit fois moins que l’écart-type du premier. Entraîner 11,2 millions de paramètres n’apporte donc *rien de mesurable* par rapport à en entraîner 5 130 — un rapport de 1 à 2 180. Deux mécanismes se compensent : avec 45 exemples par classe, le gradient est trop pauvre pour améliorer un encodeur déjà bon, et dégeler l’encodeur l’expose à une dégradation par sur-ajustement, que l’accuracy d’entraînement à 100 % (section 6.7) confirme.

Cet écart se creuse ensuite : +3,88 points en faveur du fine-tuning à 4 500 labels, +11,69 à 45 000. Le seuil où le dégel devient rentable se situe donc entre 450 et 4 500 annotations dans notre configuration. En pratique, cela signifie qu’un praticien disposant de très peu d’étiquettes a intérêt à geler son encodeur : c’est aussi précis, bien plus rapide, et le modèle tient dans quelques kilo-octets une fois l’encodeur partagé.

#### 6.5 Comparaison à la littérature

Les implémentations de référence de SimCLR rapportent, sur CIFAR-10, des évaluations linéaires de l’ordre de 94 %, avec un encodeur plus large, des lots de plusieurs milliers d’images et des pré-entraînements de l’ordre de 1 000 époques [1]. Notre évaluation linéaire plafonne à 81,87 %. L’écart n’a rien de surprenant et se lit comme un écart de moyens : nous utilisons un ResNet-18 et non un ResNet plus profond ou plus large, un lot de 256 et non de plusieurs milliers, et 100 époques et non 1 000, soit un dixième du budget de pré-entraînement. Ce qui se transfère malgré cette échelle réduite, ce n’est pas le chiffre absolu mais la *structure* du résultat : l’écart maximal en régime de faible supervision, sa disparition à supervision complète, et la primauté des augmentations de couleur. Ces trois conclusions qualitatives sont reproduites.

#### 6.6 Analyse critique : à quel coût ?

La question «le modèle est-il réellement meilleur, et à quel prix?» appelle une réponse nuancée.

- **En régime de faible supervision, oui, très largement**, et le coût supplémentaire est de 100 époques de pré-entraînement — payées *une seule fois* et amorties sur les trois régimes de labels, puisque le même encodeur est réutilisé partout.
- **En régime de supervision complète, non**. Les deux méthodes sont à parité et le pré-entraînement représente alors un coût de calcul pur, sans contrepartie mesurable.
- **Nous ne rapportons aucune comparaison de temps d’exécution**. Les mesures de durée de nos runs sont inexploitable : la machine était partagée avec d’autres charges, et une même configuration à 100 % des labels a mis 2 104 s pour la graine 123 et 11 914 s pour la graine 2026. Publier un ratio de coût à partir de ces chiffres serait malhonnête ; nous préférons ne rien affirmer que d’affirmer une mesure que le protocole ne soutient pas.

## 6.7 Surapprentissage

Dans les neuf configurations supervisées, l’accuracy d’entraînement atteint 100 %, y compris au régime de 450 labels où l’accuracy de test est de 39,52 %. Le réseau mémorise donc intégralement les 450 exemples disponibles sans en généraliser grand-chose — un écart de plus de 60 points entre train et test. C’est précisément l’espace que le pré-entraînement vient combler : il fournit à l’encodeur une structure apprise sur 50 000 images, que les 450 labels n’ont plus qu’à réorienter vers la tâche de classification.

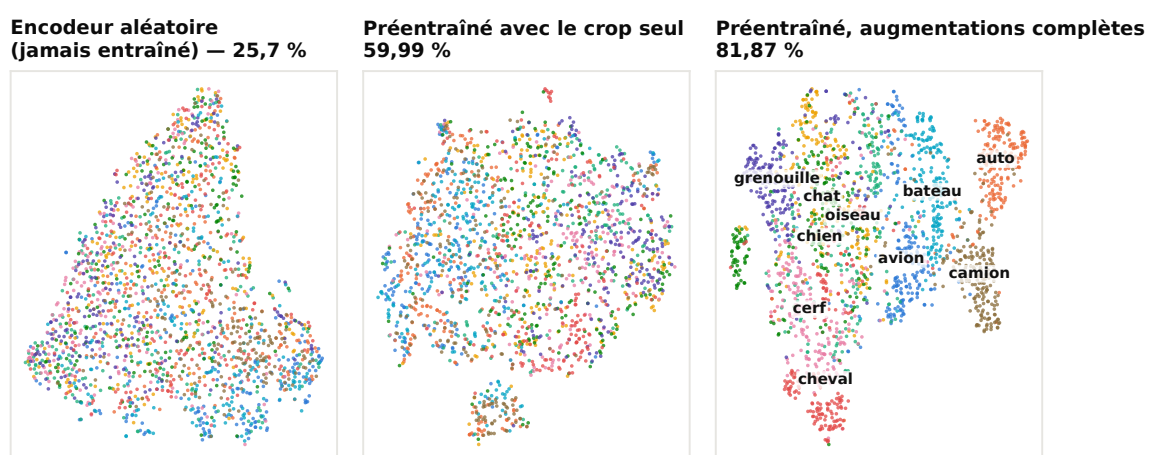
Ce constat a une conséquence méthodologique qu’il vaut mieux énoncer : à 450 labels, la baseline supervisée n’est pas limitée par sa capacité mais par son information. Augmenter la taille du modèle ne l’aiderait pas ; ajouter de la régularisation déplacerait le problème sans le résoudre. Le pré-entraînement auto-supervisé agit sur la seule variable qui manque, l’information — et il la prend là où elle est gratuite, dans les 50 000 images non étiquetées.

## 7 Déploiement : application de démonstration

Conformément à l’exigence allégée du sujet, l’application sert à *illustrer* les résultats, pas à constituer un produit fini. Elle prend la forme d’une page web autonome (`demo/index.html`) : un unique fichier HTML, sans serveur, sans dépendance réseau et sans conteneur Docker, qui s’ouvre directement dans un navigateur. Le script `scripts/preparer_demo.py` extrait les représentations, calcule les projections et embarque le tout dans la page.

La démonstration répond à la demande du sujet — « visualisation des représentations apprises, projection t-SNE/UMAP colorée par classe réelle, non utilisée à l’entraînement » — et permet de :

- comparer côte à côte les projections t-SNE et PCA de six encodeurs : l’encodeur aléatoire et les cinq configurations d’ablation ;
- colorer les 2 000 points par la classe réelle, qui n’a jamais été vue au pré-entraînement, et allumer ou éteindre chaque classe ;
- sélectionner une image et afficher ses plus proches voisins dans l’espace des représentations, ce qui rend le raccourci de couleur visible à l’œil nu sur la configuration `crop seul` ;
- consulter la matrice de confusion de l’évaluation linéaire.



2 000 images de test projetées en 2D (t-SNE) à partir de  $h \in \mathbb{R}^{512}$  · couleur = classe réelle, jamais vue au préentraînement

**FIG. 11** : Projection t-SNE des représentations  $h \in \mathbb{R}^{512}$  de 2 000 images de test, telle que la démonstration l’affiche. Les couleurs correspondent aux classes réelles, jamais utilisées au pré-entraînement ; les étiquettes sont placées au centroïde de chaque amas.

La figure 11 résume ce que la démonstration donne à voir. L’encodeur aléatoire produit un nuage informe où les couleurs se mélangent. Le pré-entraînement au recadrage seul crée une structure, mais sans séparation des classes. Les augmentations complètes font apparaître des amas nets, alignés sur les classes réelles alors qu’aucun label n’a été utilisé : c’est la preuve visuelle que la tâche contrastive a capté une structure sémantique, et non un artefact de bas niveau.

## 8 Discussion et limites

### 8.1 Limite principale : la taille du lot

Dans la perte NT-Xent (équation 1), les négatifs sont les autres vues du lot. À lot 256, chaque ancre est comparée à 510 négatifs. Les expériences de référence utilisent des lots de plusieurs milliers d’images, soit un ordre de grandeur de plus, et [1] montre que la qualité des représentations croît nettement avec ce nombre. Notre lot est dicté par la mémoire d’une carte unique de portable.

Cette limite n’a pas été mesurée : nous n’avons pas fait varier la taille du lot, faute de budget. Nous la documentons parce qu’elle plafonne mécaniquement nos chiffres absolus et interdit toute comparaison directe avec les résultats publiés. C’est aussi la piste d’amélioration la plus rentable, et celle qui relie ce projet à la section 2 : MoCo [2] découple le nombre de négatifs de la taille du lot par une file d’attente, BYOL [3] et DINO [4] suppriment le besoin de négatifs. À matériel constant, l’accumulation de gradient ne résout pas le problème — les négatifs doivent être présents *simultanément* dans le graphe de calcul.

### 8.2 Une seule graine de pré-entraînement

Toutes les incertitudes que nous rapportons proviennent des trois graines d’évaluation. Le pré-entraînement, lui, n’a tourné qu’une fois par configuration. La variabilité pré-entraînement-à-pré-entraînement n’est donc pas quantifiée. Cela n’invalide pas les écarts de plus de dix points, mais impose la prudence sur tout écart inférieur au point — d’où le traitement réservé au flip et au flou, et à l’écart de +0,43 ci-dessous.

### 8.3 Le taux d’apprentissage a été réglé à 10 %, puis transféré

Le balayage de la figure 5 a été conduit au régime 10 % des labels, et la valeur retenue (0,03) a été appliquée aux trois régimes. Rien ne garantit que 0,03 soit optimal à 100 % des labels pour la baseline supervisée ; une version antérieure de cette baseline, réglée à 0,1 sur l’ensemble du train, atteignait 94,79 %. C’est la raison pour laquelle nous parlons de **parité** à 100 % des labels et ne revendiquons pas l’écart de +0,43 point comme une supériorité du pré-entraînement. Le protocole reste équitable — la même hypothèse de transfert pèse sur les deux méthodes — mais il est moins favorable à la baseline à ce régime précis, et il faut le dire.

### 8.4 Le pré-entraînement a vu les images de validation

Le pré-entraînement contrastif a consommé les 50 000 images du train officiel, ce qui inclut les 5 000 images ensuite utilisées comme validation — *sans leurs labels*. Ce n’est pas une fuite d’étiquettes, et c’est la pratique standard en SSL, où le pré-entraînement est censé exploiter tout le corpus non annoté disponible. Nous le mentionnons pour que le lecteur puisse en juger : la validation n’a servi qu’au choix du taux d’apprentissage, et le test officiel n’a jamais été vu par le pré-entraînement.

### 8.5 Autres limites

- **Un seul jeu de données.** CIFAR-10 uniquement (section 3). Rien ne garantit que les conclusions sur les augmentations se transfèrent à une autre résolution ou à un autre domaine — notre propre résultat sur le flou en est l’illustration.

- **Une seule architecture.** ResNet-18 adapté. L'effet de la profondeur ou de la largeur de l'encodeur n'a pas été étudié.
- **Ablation additive, non factorielle** (section 5).
- **Pas de mesure de coût en temps** (section 6).

## 8.6 Pistes d'amélioration

Par ordre de rapport valeur/coût décroissant : (1) augmenter le nombre de négatifs, par une file d'attente à la MoCo plutôt que par un lot plus grand, ce que le matériel autorise; (2) allonger le pré-entraînement à 400 ou 1 000 époques, la courbe de la figure 4 n'étant pas plateau; (3) répéter le pré-entraînement sur trois graines pour quantifier l'incertitude qui manque aujourd'hui; (4) reprendre le protocole sur STL-10, dont la partition non étiquetée est conçue pour le SSL; (5) comparer à une méthode sans négatifs (BYOL) pour vérifier si l'avantage attendu se matérialise à notre échelle.

## 9 Répartition du travail

Le tableau 6 détaille les contributions de chaque membre. Les trois rôles correspondent à ceux nommés par le sujet. Le principe retenu est que **chaque membre possède le périmètre qu'il devra défendre à l'oral**. L'historique du dépôt (`git shortlog -sn`) reflète cette répartition.

**Disponibilité du code et des données.** L'intégralité du code, les fichiers de résultats et les instructions de reproduction sont publiés à l'adresse <https://github.com/sunborndev/simclr-cifar10>. Chaque ligne des tableaux 4 et 2 est reproductible par une commande unique, listée en annexe Annexe B.

## Références

- [1] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. *A Simple Framework for Contrastive Learning of Visual Representations*. ICML, 2020. <https://arxiv.org/abs/2002.05709>
- [2] K. He, H. Fan, Y. Wu, S. Xie, R. Girshick. *Momentum Contrast for Unsupervised Visual Representation Learning*. CVPR, 2020. <https://arxiv.org/abs/1911.05722>
- [3] J.-B. Grill, F. Strub, F. Altché et al. *Bootstrap Your Own Latent : A New Approach to Self-Supervised Learning*. NeurIPS, 2020. <https://arxiv.org/abs/2006.07733>
- [4] M. Caron, H. Touvron, I. Misra et al. *Emerging Properties in Self-Supervised Vision Transformers*. ICCV, 2021. <https://arxiv.org/abs/2104.14294>
- [5] K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition*. CVPR, 2016. <https://arxiv.org/abs/1512.03385>
- [6] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Rapport technique, University of Toronto, 2009. <https://www.cs.toronto.edu/~kriz/cifar.html>

## Annexe A — Hyperparamètres complets

Toutes les valeurs ci-dessous sont celles des exécutions publiées; elles sont également sérialisées dans les fichiers `resultats.json` du dépôt, ce qui permet de les vérifier sans relire le code.

## Annexe B — Commandes de reproduction

Les commandes suivantes régénèrent chaque résultat publié. Elles supposent un environnement Python conforme à `requirements.txt` et sont lancées depuis la racine du dépôt.

TAB. 6 : Répartition des rôles et des contributions.

| Membre            | Rôle (sujet)                  | Contributions précises                                                                                                                                                                                                                                                                                 | Sections            |
|-------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| KAMBIA Rafiatou   | Data / Experiment Engineer    | Chargement et analyse exploratoire de CIFAR-10; chaîne d'augmentations et sa paramétrisation par brique; découpage stratifié train/validation et script de vérification anti-fuite; balayage symétrique du taux d'apprentissage; documentation de reproduction (REPRODUCTION.md), gestion des graines. | 3, 4.2, 4.8, 4.9, 8 |
| ALLAGLO Kossiko   | Model / Research Engineer     | Encodeur ResNet-18 adapté et tête de projection; implémentation de la perte NT-Xent et de sa vérification; boucle de pré-entraînement (échauffement, cosinus, précision mixte); les trois protocoles d'évaluation et le témoin aléatoire; conception et exécution de l'étude d'ablation.               | 4.1, 4.3–4.7, 5, 6  |
| AMEGNRAN Athanase | Reporting / Backend Developer | Application web de démonstration (extraction des représentations, projections t-SNE et PCA, plus proches voisins, matrice de confusion); README et documentation d'installation; figures; assemblage du rapport et de la présentation.                                                                 | 1, 2, 7, 9          |

**TAB. 7 :** Pré-entraînement contrastif (scripts/pretrainer\_experimental.py).

| Paramètre            | Valeur                                                   | Option              |
|----------------------|----------------------------------------------------------|---------------------|
| Données              | 50 000 images du train officiel, sans labels             | --data-dir          |
| Architecture         | ResNet-18 adapté ( $3 \times 3$ pas 1, sans max pooling) | —                   |
| Représentation       | $h \in \mathbb{R}^{512}$                                 | —                   |
| Tête de projection   | $512 \rightarrow 512 \rightarrow 128$ , ReLU             | —                   |
| Époques              | 100                                                      | --epochs            |
| Taille de lot        | 256 (soit 512 vues, 510 négatifs par ancre)              | --batch-size        |
| Optimiseur           | AdamW                                                    | —                   |
| Taux d'apprentissage | $3 \cdot 10^{-4}$                                        | --learning-rate     |
| Taux minimal         | $10^{-6}$                                                | --min-learning-rate |
| Échauffement         | 10 époques, linéaire                                     | --warmup-epochs     |
| Décroissance         | cosinus sur les 90 époques restantes                     | —                   |
| Weight decay         | $10^{-4}$                                                | --weight-decay      |
| Température $\tau$   | 0,5                                                      | --temperature       |
| Précision mixte      | activée (torch.amp)                                      | —                   |
| Graine               | 42                                                       | --seed              |

**TAB. 8 :** Les trois protocoles d'évaluation aval. Le témoin à encodeur aléatoire suit exactement le protocole de l'évaluation linéaire.

|                       | Évaluation linéaire                     | Fine-tuning                     | Supervisé                       |
|-----------------------|-----------------------------------------|---------------------------------|---------------------------------|
| Encodeur              | gelé                                    | entraîné                        | entraîné                        |
| Poids entraînaibles   | $\approx 5\,130$                        | $\approx 11,2$ M                | $\approx 11,2$ M                |
| Entrée du classifieur | $h$ standardisé                         | $h$                             | $h$                             |
| Époques               | 50                                      | 100                             | 100                             |
| Taille de lot         | 256                                     | 256                             | 256                             |
| Optimiseur            | SGD, momentum 0,9                       | SGD, momentum 0,9, Nesterov     | SGD, momentum 0,9, Nesterov     |
| Taux d'apprentissage  | 0,1                                     | 0,03                            | 0,03                            |
| Décroissance          | cosinus $\rightarrow lr \times 10^{-3}$ | cosinus $\rightarrow 10^{-4}$   | cosinus $\rightarrow 10^{-4}$   |
| Weight decay          | —                                       | $5 \cdot 10^{-4}$               | $5 \cdot 10^{-4}$               |
| Augmentations         | aucune                                  | RandomCrop(32, pad=4)<br>+ Flip | RandomCrop(32, pad=4)<br>+ Flip |
| Graines               | 42, 123, 2026                           | 42, 123, 2026                   | 42, 123, 2026                   |

**TAB. 9 :** Correspondance résultat → commande → sortie.

| Résultat                                               | Commande                                                                                                                            | Sortie                                             |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Pré-entraînement principal                             | <code>python -m scripts.preentrainer_experimental --augmentations complet --epochs 100</code>                                       | <code>outputs/simclr_cifar10_100ep/</code>         |
| Tableau 4, lignes <i>linear probe</i> et <i>témoin</i> | <code>python -m scripts.experimentier_fractions_labels --seeds 42 123 2026 --evaluer-sur test</code>                                | <code>outputs/experience_labels_45k/</code>        |
| Tableau 4, ligne <i>fine-tuning</i>                    | <code>python -m scripts.experimentier_supervise_fractions --checkpoint &lt;ckpt&gt; --learning-rate 0.03 --seeds 42 123 2026</code> | <code>outputs/finetuning_simclr_45k/</code>        |
| Tableau 4, ligne <i>supervisé</i>                      | <code>python -m scripts.experimentier_supervise_fractions --learning-rate 0.03 --seeds 42 123 2026</code>                           | <code>outputs/supervise_from_scratch_45k/</code>   |
| Tableau 2 (intégral)                                   | <code>python -m scripts.reproduire_ablation</code>                                                                                  | <code>outputs/ablation/tableau_ablation.csv</code> |
| Figure 5 (balayage)                                    | <code>python -m scripts.experimentier_supervise_fractions --learning-rate &lt;lr&gt; --evaluer-sur validation</code>                | <code>outputs/*_recherche_lr/</code>               |
| Contrôle anti-fuite                                    | <code>python -m scripts.inspecter_split_validation</code>                                                                           | VERDICT :<br>protocole propre                      |
| Application de démonstration                           | <code>python -m scripts.preparer_demo</code>                                                                                        | <code>demo/index.html</code>                       |
| Figures du rapport                                     | <code>python tools/fig_resultats.py puis tools/fig_schemas.py</code>                                                                | <code>docs/rapport/fig/</code>                     |

## Annexe C — Balayage du taux d'apprentissage, chiffres détaillés

Accuracy sur les 5 000 images de validation, au régime 10 % des labels, graine 42. Ces chiffres sont ceux tracés en figure 5.

**TAB. 10 :** Balayage symétrique. La même grille a été appliquée aux deux méthodes.

| Taux d'apprentissage                  | 0,003 | 0,01  | <b>0,03</b>  | 0,1    | 0,2   |
|---------------------------------------|-------|-------|--------------|--------|-------|
| Fine-tuning SimCLR                    | 81,60 | 81,44 | <b>83,52</b> | 82,14  | 63,46 |
| Supervisé <i>from scratch</i>         | 60,60 | 65,40 | <b>70,46</b> | 68,84  | 66,64 |
| Accuracy d'entraînement (fine-tuning) | 92,36 | 99,73 | 100,00       | 100,00 | 96,40 |
| Accuracy d'entraînement (supervisé)   | 84,42 | 99,98 | 99,98        | 100,00 | 94,42 |

Les deux dernières lignes éclairent la forme des courbes. À 0,003, aucune des deux méthodes n'a fini de converger en 100 époques — l'accuracy d'entraînement n'atteint pas 100 %. À 0,2, elle redescend également : l'optimisation devient instable. Entre les deux, les modèles atteignent la mémorisation complète et seule la généralisation les départage. L'optimum commun à 0,03 n'est donc pas un artefact de convergence.

## Annexe D — Coût de calcul

**TAB. 11 :** Coût des cinq pré-entraînements, sur une NVIDIA GeForce RTX 4060 Laptop. La colonne « durée totale observée » est donnée pour la transparence, non pour la comparaison : voir la mise en garde ci-dessous.

| Configuration     | Époques | Médiane s/époque | Coût implicite | Durée totale observée |
|-------------------|---------|------------------|----------------|-----------------------|
| crop seul         | 100     | 32,7             | ≈ 55 min       | 914 min               |
| crop + flip       | 100     | 31,5             | ≈ 53 min       | 53 min                |
| + color jitter    | 100     | 33,2             | ≈ 55 min       | 457 min               |
| + niveaux de gris | 100     | 30,4             | ≈ 51 min       | 92 min                |
| + flou gaussien   | 100     | 29,7             | ≈ 50 min       | 120 min               |

Deux enseignements, et une mise en garde.

**La chaîne d'augmentations ne coûte rien.** Les médianes s'échelonnent de 29,7 à 33,2 secondes par époque, soit moins de 12 % d'écart entre la configuration la plus légère et la plus lourde. Le coût du pré-entraînement est dominé par les passes avant et arrière du ResNet, pas par les transformations d'images. Les cinq configurations de l'ablation ont donc bien reçu, en pratique comme en principe, le même budget de calcul.

**Le pic mémoire est identique partout** — 1 376 Mo, à l'octet près pour les cinq runs. C'est cohérent : la taille de lot et l'architecture sont fixes, et seules les transformations, effectuées sur CPU, changent.

**Mise en garde sur les durées totales.** Les durées totales observées vont de 53 minutes à 914 minutes pour des configurations dont le coût réel est identique à 12 % près. La machine était partagée avec d'autres charges pendant plusieurs runs, et quelques époques isolées ont duré des dizaines de minutes. C'est la raison pour laquelle nous ne publions *aucune* comparaison de coût en temps entre méthodes (section 8) : la médiane par époque est la seule mesure de coût que notre protocole soutient.

**Budget total.** En s'appuyant sur les médianes, les cinq pré-entraînements représentent environ 4,4 heures de GPU. En y ajoutant les 36 exécutions d'évaluation ( $4 \text{ bras} \times 3 \text{ fractions} \times 3 \text{ graines}$ ) et les 10 exécutions du balayage du taux d'apprentissage, le budget total du projet se situe aux alentours de sept heures de calcul.

## Annexe E — Nomenclature

| Symbole ou terme           | Signification                                                                                    |
|----------------------------|--------------------------------------------------------------------------------------------------|
| $x$                        | une image du jeu de données, sans son étiquette                                                  |
| $\mathcal{T}$              | la famille d'augmentations aléatoires (recadrage, couleur, flou...)                              |
| $t, t'$                    | deux transformations tirées indépendamment dans $\mathcal{T}$                                    |
| $\tilde{x}_i, \tilde{x}_j$ | les deux vues augmentées de $x$ ; elles forment une <i>paire positive</i>                        |
| $f(\cdot)$                 | l'encodeur — le ResNet-18 adapté, seul élément réutilisé en aval                                 |
| $h \in \mathbb{R}^{512}$   | la représentation produite par l'encodeur                                                        |
| $g(\cdot)$                 | la tête de projection — un MLP, jeté après le pré-entraînement                                   |
| $z \in \mathbb{R}^{128}$   | la projection sur laquelle la perte contrastive s'applique                                       |
| $\tau$                     | la température de NT-Xent ; contrôle la dureté de la discrimination                              |
| $N$                        | la taille de lot ; le lot fournit $2N - 1$ candidats et $2N - 2$ négatifs par ancre              |
| <i>tâche prétexte</i>      | la tâche artificielle résolue sans étiquettes (ici : reconnaître les deux vues d'une même image) |
| <i>tâche aval</i>          | la vraie tâche visée (ici : classer en 10 catégories)                                            |
| <i>évaluation linéaire</i> | protocole où l'encodeur est gelé et où seule une couche linéaire s'entraîne                      |
| <i>fine-tuning</i>         | protocole où l'encodeur et le classifieur s'adaptent ensemble                                    |
| <i>ablation</i>            | retrait contrôlé d'un composant, tout le reste étant maintenu identique                          |
| <i>témoin</i>              | bras de contrôle servant à attribuer un effet à sa cause                                         |